3 Module 1 — Course Framing & the Agent Design Space

</> Code

What an agent is, what 'agentic' means, and why the simplest architecture usually wins

3.1 Where this module sits

This is the intellectual on-ramp for the whole course. Everything after this — reasoning patterns, tool use, memory, multi-agent coordination, anti-patterns — is a refinement of three ideas introduced here:

1. What an agent actually *is*, mechanically.
2. What makes a system "agentic," and why that is a spectrum, not a label.
3. Why adding agency is a *cost*, and how to decide when the cost is worth paying.

By the end you should be able to look at any "AI agent" system and describe it precisely: what its control flow is, what patterns it stacks, and whether a simpler design would have done the same job. The hands-on counterpart to this module is Lab 1, where you build a working coding agent and watch the difference between *telling it the steps* and *letting it decide* with your own eyes.

3.2 What is an agent?

Start with the smallest true statement and build up.

An **agent** is a **large language model running in a loop, with memory and tools, pursuing a goal**.

That sentence has four load-bearing words. Take them one at a time.

3.2.1 The LLM is the engine

The LLM is the only component that makes decisions. It cannot do anything on its own — it reads text and writes text — but the text it writes can be *interpreted by the surrounding program as instructions*: "call this tool," "I am done," "here is the answer." Everything else in an agent exists to feed the model good input and to act on its output.

3.2.2 Tools are how it touches the world

A bare LLM can only produce text. A **tool** is a function the surrounding program exposes to the model — search the web, run code, query a database, write a file. The model does not execute the tool; it *requests* the call by

emitting a structured message, the program runs the function, and the result is fed back into the model's context. Tools are the agent's hands and senses.

In this course tools are delivered through the **Model Context Protocol (MCP)** — a standard way to expose a catalog of tools to any model. MCP is covered in depth in the MCP primer; for now, treat it as "the USB-C port for tools." In Lab 1 your agent will get exactly one MCP tool — a Python sandbox — and that single tool is enough to make it genuinely capable.

3.2.3 Memory is what survives the loop

The LLM itself is stateless: each call is independent. **Memory** is anything the program carries between calls so the agent can build on prior work. The simplest memory is the running conversation transcript (the *scratchpad*). A richer form is **retrieval-augmented generation (RAG)**: store documents in a vector database, and at each step retrieve the few most relevant chunks and paste them into the context. RAG is "memory the agent can look things up in." (Memory design is its own module — Module 5 — and RAG mechanics are in the retrieval primers.)

3.2.4 The loop is what makes it an agent

A single LLM call answers once and stops. An agent **loops**: think → act (call a tool) → observe the result → think again → … → decide it is finished. The loop is the difference between a model that *answers* and a system that *gets something done*. It is also where the danger lives — a loop with no exit condition is the first anti-pattern in this course (Module 12, *No Termination Criteria*).

3.2.5 The constitutional prompt is the agent's character

One more piece, easy to underrate. The **constitutional prompt** (often called the system prompt or persona) is a standing instruction prepended to every model call that defines *how the agent should behave*: its role, its standards, what it must never do, how it should handle uncertainty. It is not the task — the task changes every run; the constitution does not.

The constitutional prompt makes a *dramatic* difference. The same model, same tools, same task, behaves like a careless intern or a meticulous senior engineer depending almost entirely on this text. In Lab 1 you will use a real "Expert Coder" constitution that, among other things, forbids the agent from producing stub code or confabulating library APIs — and you will see it visibly hold the model to a higher standard. We will return to constitutional prompts in every module; they are the cheapest, highest-leverage control you have.

> ⓘ **The anatomy, in one sentence**
>
> An agent is an **LLM** (the decider), shaped by a **constitutional prompt** (its character), running in a **loop** (think–act–observe), using **tools** (hands) and **memory** (continuity), to pursue a **goal**.

The figure below shows the loop. Read it as: the constitution and goal seed the context; the model decides; if it asks for a tool, the program runs it and feeds the result back; otherwise the loop ends with an answer.

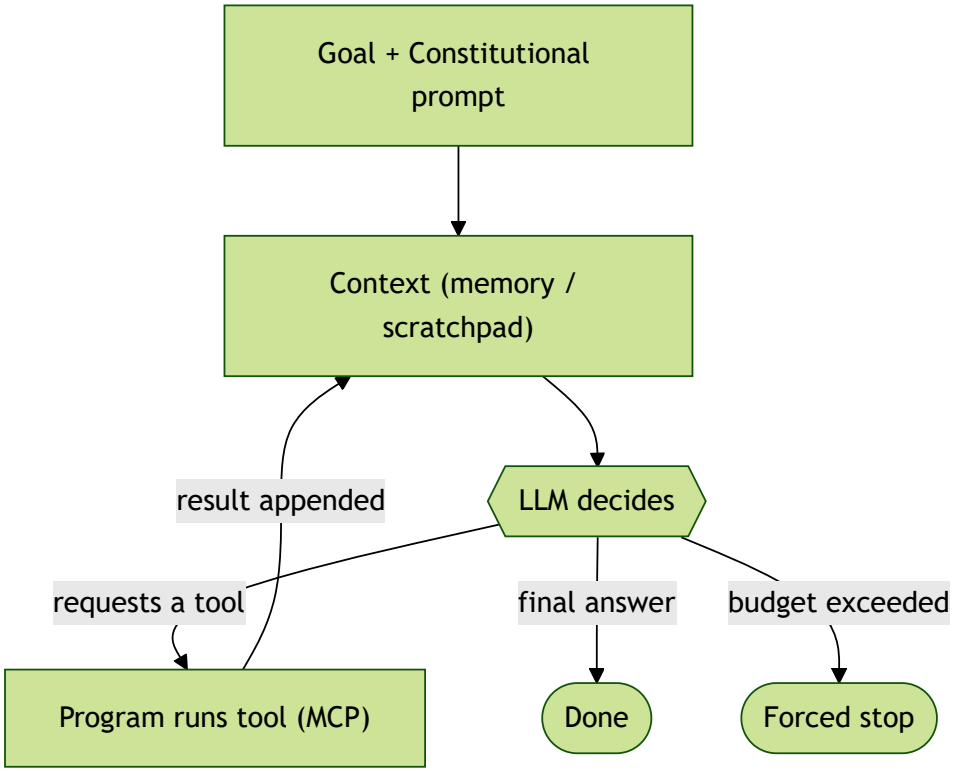


Figure 3.1: The agent loop. The model is the only decision-maker; tools and memory feed it; the loop continues until the model emits a final answer or a budget is hit.

3.3 What “agentic” means

People use “agent” as a marketing word. We will use it as an *engineering* word, and the distinction that makes it precise is **who controls the flow of execution**.

- **Predetermined control flow**: a human wrote, in code, the sequence of steps. The LLM fills in steps but never decides *what happens next*. This is a **workflow**.
- **Model-driven control flow**: the LLM decides *what happens next* — which tool to call, whether to loop again, when it is done. This is an **agent**.

That is the entire definition. “Agentic” is the degree to which the *model*, rather than your code, is steering. It is not about how many LLM calls you make or how impressive the diagram looks.

3.3.1 Workflow vs. agent (Anthropic’s framing)

Anthropic’s widely-cited framing draws exactly this line: a **workflow** orchestrates LLMs and tools through *predefined code paths*; an **agent** lets the LLM *direct its own process and tool use*. Neither is better. They trade the same currency in opposite directions:

	Workflow (predetermined)	Agent (model-driven)
Control flow	Fixed by your code	Decided by the model
Predictability	High	Lower
Flexibility on novel inputs	Low	High
Debuggability	Easy (you can read the path)	Hard (the path is emergent)
Failure modes	Few, known	Many, some surprising
Right when…	The steps are known in advance	The steps depend on what is discovered along the way

The practical rule, which we will defend all term: **if you can write the steps down, write them down.** Reach for an agent only when the steps genuinely cannot be known until the work is underway.

3.3.2 The spectrum of agency

"Workflow vs. agent" is a useful dividing line but the real picture is a spectrum. Each step right adds capability *and* cost.

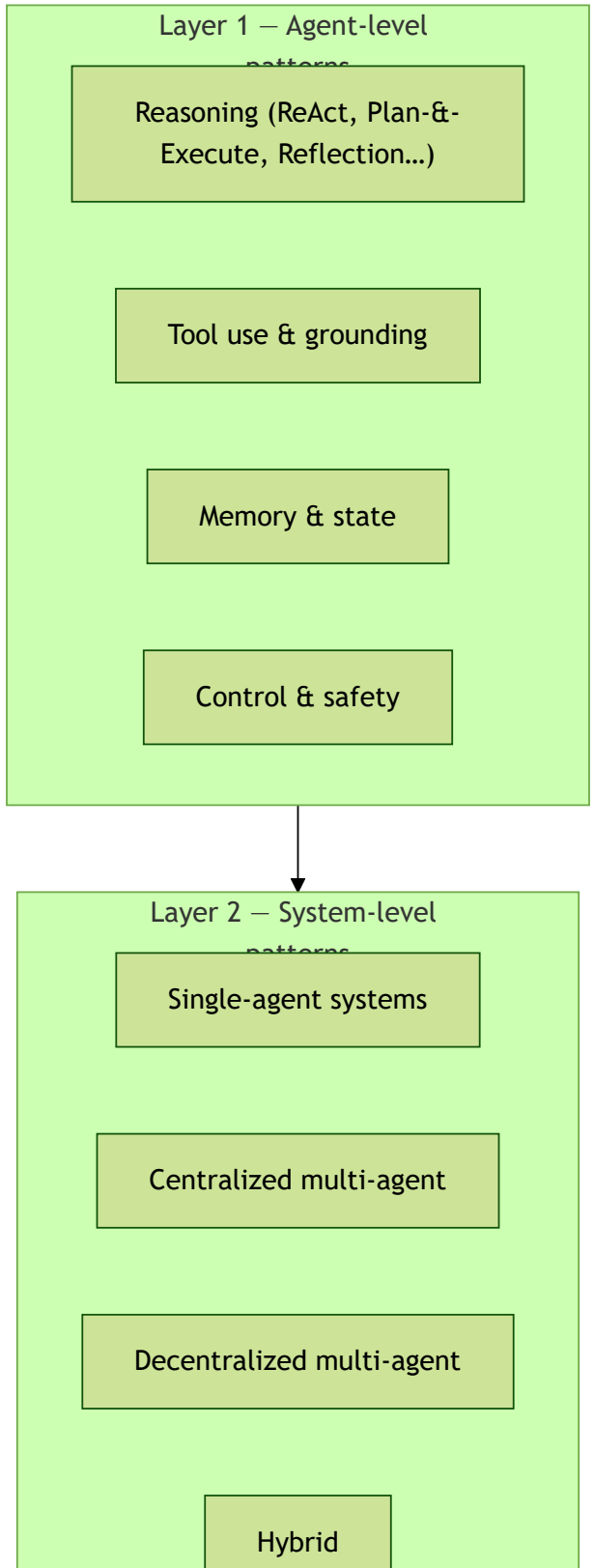
Figure 3.2: The spectrum of agency. Capability and autonomy rise left to right; so do latency, token cost, and the number of failure modes.

- **Single LLM call** — one prompt, one answer. No tools, no loop. Astonishingly often, this is enough.
- **Tool-augmented LLM** — one call that may invoke a tool (e.g., a calculator), but you control the flow.
- **Workflow** — multiple LLM/tool steps wired together by *your* code.
- **Single agent** — the model drives its own loop over a tool catalog.
- **Multi-agent system** — several agents coordinate; now you also own a coordination problem.

The mistake almost every newcomer makes is to start on the right and justify it backward. The discipline this course builds is to start on the *left* and move right only under pressure.

3.4 The three-layer model of this course

Everything we study fits into three layers. Keep this map in your head; every later module is a deeper pass over one band of it.


Layer 1 — Agent-level patterns
Reasoning (ReAct, Plan-&-Execute, Reflection…)
Tool use & grounding
Memory & state
Control & safety
Layer 2 — System-level patterns
Single-agent systems
Centralized multi-agent
Decentralized multi-agent
Hybrid

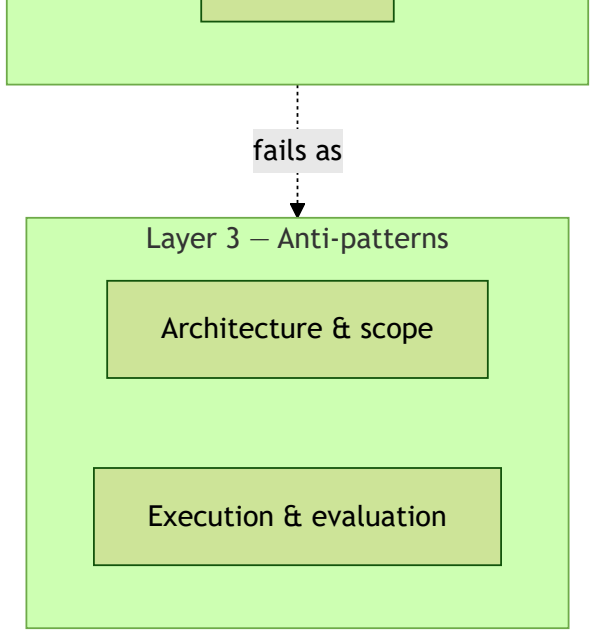


Figure 3.3: The three-layer course model. Layer 1 patterns compose inside Layer 2 systems; Layer 3 anti-patterns are the characteristic ways both fail.

- **Layer 1 — agent-level patterns**: the building blocks *inside* one agent — how it reasons, uses tools, remembers, and stays safe.
- **Layer 2 — system-level patterns**: how agents (one or many) are *composed* into a system.
- **Layer 3 — anti-patterns**: the recurring ways Layer 1 and Layer 2 designs fail. We treat anti-patterns as *diagnostics* — symptom → cause → fix — not as a list of scoldings.

This is the **pattern stack** diagram, and it recurs in every module. When we study a real system we will draw it as a stack: which Layer 1 patterns it uses, composed by which Layer 2 pattern, and which Layer 3 failure it is one bad day away from. Learning to *read a system as a pattern stack* is the single most transferable skill in this course.

3.5 The two principles that govern every design choice

3.5.1 The escalation principle: simplest viable architecture first

Begin with the simplest point on the spectrum that could plausibly work. Escalate one step only when you can name the *specific* way the simpler design failed. “It might not handle edge cases” is not a reason; “here are three transcripts where the workflow took the wrong branch because the branch depends on information we only get mid-task” is a reason.

This is not minimalism for its own sake. It is that **complexity must be earned by demonstrated failure of something simpler**, because every step up the spectrum is a step you will have to debug, evaluate, and pay for — forever.

3.5.2 The cost-of-every-pattern lens

There is no free pattern. Every step toward more agency buys capability with three currencies:

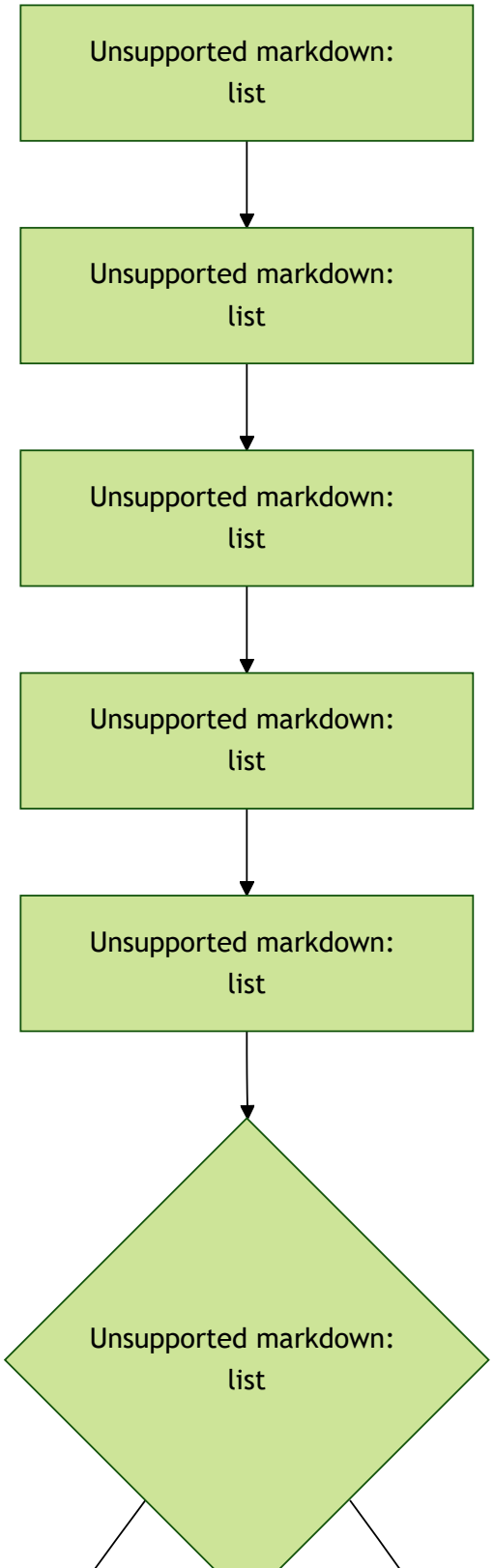
- **Latency** — more model round-trips, slower responses.
- **Tokens (money)** — every loop iteration re-sends the growing context.
- **Failure modes** — each pattern adds *characteristic* ways to fail. Reflection can talk itself out of a correct answer. Multi-agent debate can collapse into agreement. An unbounded loop can burn your budget at 3 a.m.

Whenever we introduce a pattern in this course, we will immediately ask: *what does it cost, and what new way can it fail?* A pattern you cannot price is a pattern you cannot responsibly deploy.

3.6 How to read an agentic system

Put the pieces together into a procedure you can run on any system — a paper, a blog post, a product, your own design:

1. **Find the control flow.** Who decides what happens next at each step — the code or the model? That places it on the spectrum.
2. **Name the Layer 1 patterns.** How does each agent reason, use tools, remember, stay bounded?
3. **Name the Layer 2 composition.** Single agent? Orchestrator-worker? Pipeline? Debate?
4. **Draw the pattern stack.** Layer 1 blocks inside the Layer 2 box.
5. **Price it.** Where are the latency, token, and failure costs?
6. **Ask the escalation question.** Could a simpler point on the spectrum have done this job? If yes, the extra agency is unjustified until proven otherwise.


Unsupported markdown:
list
Unsupported markdown:
list
Unsupported markdown:
list
Unsupported markdown:
list
Unsupported markdown:
list
Unsupported markdown:
list

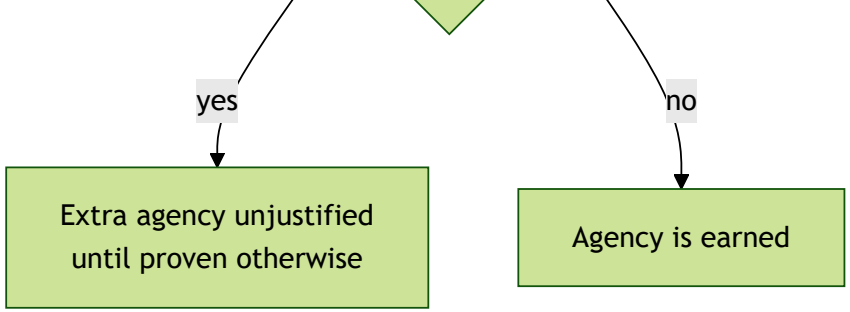


Figure 3.4: The six-step procedure for reading any agentic system: locate the control flow, name the Layer 1 patterns and Layer 2 composition, draw the pattern stack, price it, then ask whether a simpler point on the spectrum would have sufficed.

You will run this procedure dozens of times before the term ends. By Module 15 you will do it on a problem you have never seen, on the spot.

3.7 One problem, four ways (a thought experiment)

To make the spectrum concrete, take a single mundane problem and look at it through four lenses. **Customer-support triage**: an email arrives; the system should either answer it, route it to the right team, or escalate to a human.

Before reading the table, decide for yourself which design you would ship.

Design	How it works	Wins	Pays
Single LLM call	One prompt: "classify this email and draft a reply."	Cheapest, fastest, trivially debuggable.	Cannot look anything up; wrong on anything needing account data.
Workflow	Code path: classify → *if billing*, fetch account via tool → draft reply → *if low confidence*, route to human.	Predictable, auditable, still cheap. Handles the 90% you can foresee.	Brittle on inputs you did not foresee; every new case is a code change.
Single agent	Model gets tools (lookup, knowledge base, ticketing) and decides which to use and when to stop.	Handles novel, multi-step cases without new code.	Slower, pricier per ticket, harder to debug; needs a step/budget cap.
Multi-agent	A classifier agent routes to specialist agents	Genuine specialization; parallelism.	Coordination tax: more calls, more latency, many

Design	How it works	Wins	Pays
	(billing, technical, retention), a supervisor merges.		more failure modes.

The intended realization: for *this* problem, the workflow is almost certainly the right answer, and the multi-agent design is the one that looks most impressive in a slide and is the worst engineering decision. The point of the course is not that agents are bad — it is that **agency is a design choice you must justify**, and the instinct to reach for the most agentic option is exactly the instinct we are here to invert.

We deliberately keep this conceptual. You will *build* the workflow-vs-agent contrast yourself in Lab 1, on a problem with a single, checkable right answer, so the difference is not a story but something you measured.

3.8 Connections and self-study

- **Lab 1 —** Build a Coding Agent: the hands-on counterpart. You build an Expert Coder agent and run it two ways on the same task.
- **Primer —** MCP: the tool protocol used in every lab.
- **Primer —** Multi-agent design patterns: previews Layer 2; revisited in Modules 7–10.
- The pattern-stack diagram returns in **every** subsequent module — invest in it now.

3.9 Self-check

You understand this module if you can answer these without notes:

1. In one sentence each, what do the LLM, tools, memory, the loop, and the constitutional prompt contribute to an agent?
2. Give the one-line test that distinguishes a workflow from an agent.
3. Name the five points on the spectrum of agency, in order.
4. State the escalation principle and explain why “it might fail on edge cases” is not a valid reason to escalate.
5. Name the three currencies every pattern spends.
6. Pick any AI system you have used and run the six-step “how to read an agentic system” procedure on it out loud.